

Python

```
from abc import ABC, abstractmethod
```

```
# Abstract Class: Cannot be instantiated
```

```
class PaymentProcessor(ABC):
```

```
    # Concrete Method (has implementation)
```

```
    def process_transaction(self, amount):
```

```
        print(f"--- Processing ${amount} transaction ---")
```

```
        return self._authorize_payment(amount) # Calls the abstract method
```

```
    # Abstract Method: Must be implemented by subclasses
```

```
    @abstractmethod
```

```
    def _authorize_payment(self, amount):
```

```
        pass # No implementation here
```

```
# Concrete Subclass: Must implement the abstract method
```

```
class CreditCardProcessor(PaymentProcessor):
```

```
    def _authorize_payment(self, amount):
```

```
        # Real-world: Connects to a payment gateway API
```

```
        print("Authorizing via credit card gateway...")
```

```
        if amount < 500:
```

```
            return "SUCCESS: CC authorized"
```

```
        return "FAILURE: CC limit exceeded"
```

```
# Example Usage
```

```
# processor = PaymentProcessor() # TypeError: Can't instantiate abstract class
```

```
cc_proc = CreditCardProcessor()
```

```
status = cc_proc.process_transaction(450.00)
```

```
print(f"Transaction Status: {status}")
```

```
# If CreditCardProcessor failed to implement _authorize_payment,
```

```
# Python would raise a TypeError when trying to instantiate it.
```

8. Errors, Exceptions, and File Handling

8.1 Errors and Exceptions

Syntax Errors (Compile-time)

A **Syntax Error** occurs when the code violates the grammatical rules of the Python language. The interpreter cannot parse the code and will stop execution before the program even starts.

Real-time Example: Missing a colon, parenthesis, or having improper indentation.

Python

```
# --- Syntax Error Example ---
```

```
# if 5 > 2    <-- SyntaxError: expected ':'
```

```
# print("Five is greater than two")
```

```
# print("Hello" <-- SyntaxError: unclosed parenthesis
```

Logical Errors and Runtime Errors (Exceptions)

1. **Logical Errors:** The code runs without crashing, but the output is incorrect because the logic is flawed. These are the hardest to debug.
 - *Example:* Calculating area as $A = L + W$ instead of $A = L \times W$. The program runs but gives the wrong area.
2. **Runtime Errors (Exceptions):** The program starts executing but encounters an unexpected situation (like dividing by zero, or trying to access a file that doesn't exist) that forces it to terminate abruptly. These are called **Exceptions**.

Common Built-in Exceptions

Exception	Description	Real-time Example
TypeError	An operation or function is applied to an object of an inappropriate type.	'5' + 10 (Adding string and integer)
ValueError	A function receives an argument of the correct type but an inappropriate value.	int('hello') (Cannot convert string to integer)
ZeroDivisionError	Occurs when dividing a number by zero.	10 / 0
IndexError	A sequence index (string, list, tuple) is out of range.	my_list = [10]; my_list[1]
KeyError	Trying to access a dictionary key that doesn't exist.	my_dict = {'a': 1}; my_dict['b']
FileNotFoundError	Trying to open a file that does not exist.	open('nonexistent.txt', 'r')
AttributeError	Trying to access an attribute or method that an object does not have.	'hello'.append('x') (Strings don't have an append method)

8.2 Exception Handling

Exception handling allows a program to gracefully manage runtime errors instead of crashing.

The try, except, else, and finally blocks

- **try:** The code that might raise an exception is placed here.
- **except:** If an exception occurs in the try block, the code in the except block is executed.
- **else:** (Optional) Code that executes **ONLY** if the try block completes **without** raising an exception.
- **finally:** (Optional) Code that executes **always**, regardless of whether an exception occurred or was handled. Often used for cleanup operations.

Python

Real-time Example: Robust User Input Handling

```
def get_positive_integer(prompt):
    while True:
        try:
            # 1. Code that might fail (convert input to int)
            user_input = input(prompt)
            number = int(user_input)

            # 2. Logical check that might raise an error
            if number <= 0:
                # 3. Raising an Exception (see next topic)
                raise ValueError("Input must be a positive number.")

        except ValueError as e:
            # 4. Handle specific exception
            print(f"Invalid input: {e}. Please try again.")

        except Exception as e:
            # Catching any other unexpected error
            print(f"An unexpected error occurred: {e}")

        else:
            # Executes if NO exception was raised in try block
            print("Input successfully validated.")
            return number

    finally:
        # Executes always, cleanup/logging can go here
        print("--- Input attempt finished ---")

# quantity = get_positive_integer("Enter quantity: ")
# print(f"You entered: {quantity}")
```

Handling Specific Exceptions

It's best practice to handle specific exceptions rather than a general Exception. This prevents catching unexpected errors (like SystemExit) and gives you context-specific recovery logic.

Python

```
data = {'apple': 1.5, 'banana': 0.5}
price = 0

try:
    item_name = 'orange'
    # This might raise KeyError
    price = data[item_name]

    # This might raise ZeroDivisionError
    ratio = 10 / price

except KeyError:
    print(f"Error: Product '{item_name}' not found in catalog.")
    price = 0.0 # Set default value

except ZeroDivisionError:
    print("Error: Price cannot be zero for division.")
```

```
except Exception as e:
    # Fallback for any other unexpected error
    print(f'An unknown error occurred: {e}')

# This will print: Error: Product 'orange' not found in catalog.
```

Raising Exceptions (raise keyword)

You can manually trigger an exception using the `raise` keyword. This is useful for enforcing business rules or signaling errors from within functions.

```
Python
def validate_age(age):
    if not isinstance(age, int):
        raise TypeError("Age must be an integer.")
    if age < 0 or age > 120:
        raise ValueError("Age must be between 0 and 120.")
    return age

try:
    # validate_age("twenty") # Raises TypeError
    valid_age = validate_age(150) # Raises ValueError
except ValueError as e:
    print(f"Validation Error: {e}")
```

User-Defined Exceptions (Basic)

For complex applications, you can create your own custom exceptions by defining a new class that inherits from the base `Exception` class (or one of its specialized subclasses).

```
Python
class InsufficientFundsError(Exception):
    """Custom exception raised when a withdrawal exceeds the balance."""
    def __init__(self, requested, available):
        self.requested = requested
        self.available = available
        super().__init__(f'Requested {requested}, but only {available} available.')

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(amount, balance)
    return balance - amount

# Real-time usage
current_balance = 500
try:
    new_balance = withdraw(current_balance, 700)
    print(f'New Balance: {new_balance}')
except InsufficientFundsError as e:
    print(f'Transaction failed: {e}')
```

8.3 File Handling

File handling involves reading data from external files and writing data to them.